



中国科学院大学
University of Chinese Academy of Sciences



计算机体系结构铁人三项

题 目： SoomRV 设计文档
成 员： 刘卓敏 卫佳乐 陈冠宇 刘元昊 何国真
日 期： 2025.03.08

SoomRV 设计文档

1 项目背景与需求分析

SoomRV 的目标不是单点性能冲刺，而是在一个可综合、可回归的 RV32 核上同时满足四个条件：

1. 每周期最多 4 条指令的乱序执行能力。
2. 分支预测具备可恢复性，且在 Linux 场景下误判代价可控。
3. 提交路径满足精确异常与按序可见性。
4. 整个实现能在仿真环境中稳定启动 Linux，并提供可量化性能数据。

把这四点放在同一工程里，核心矛盾其实很清楚：预测器想尽快学习，但错误路径训练会污染长期行为；调度器希望激进发射，但提交和异常恢复必须保留顺序语义。

因此本项目采用了两个策略：

- 分支预测拆分为“目标预测 (BTB/RET) + 方向预测 (TAGE)”，并把方向训练放到提交侧，降低错误路径污染。
- 数据面按 4 宽指令流组织 (Decode/Rename/Commit 均为 `DEC_WIDTH=4`)，执行面按 5 个端口并行 (3 ALU + 2 AGU)，通过标签与序号机制维持 OoO 正确性。

2 总体设计

SoomRV 采用“4 宽进入、乱序执行、按序提交”的组织方式：`DEC_WIDTH=4` 决定了解码/重命名/提交宽度，执行侧由 `NUM_ALUS=3` 与 `NUM_AGUS=2` 组成 5 端口执行面。该宽度配置在 `src/Config.sv:L20,L94-L100` 定义，并被 `Core` 顶层串接到前后端模块。

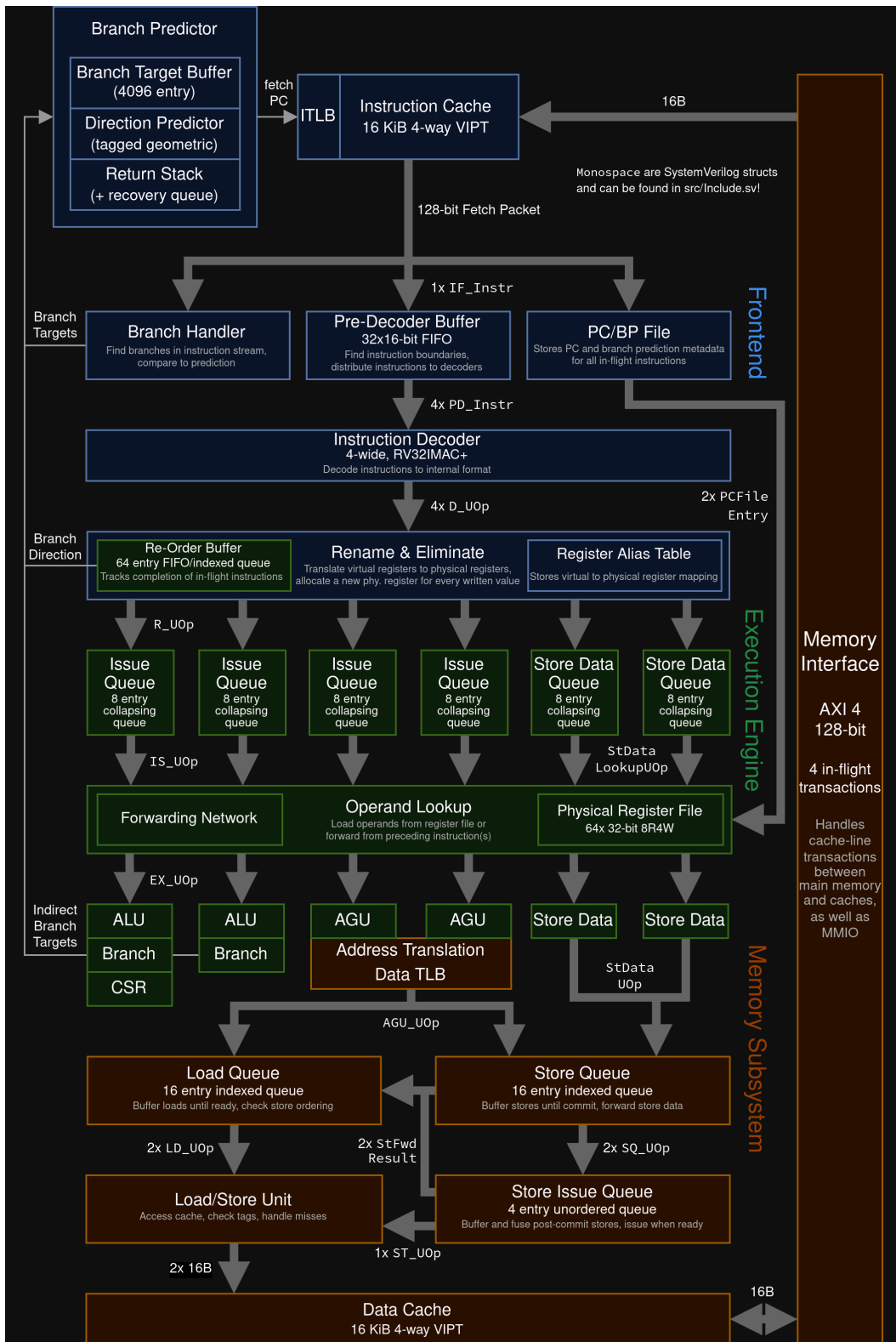


图 2-1 SoomRV 项目架构图

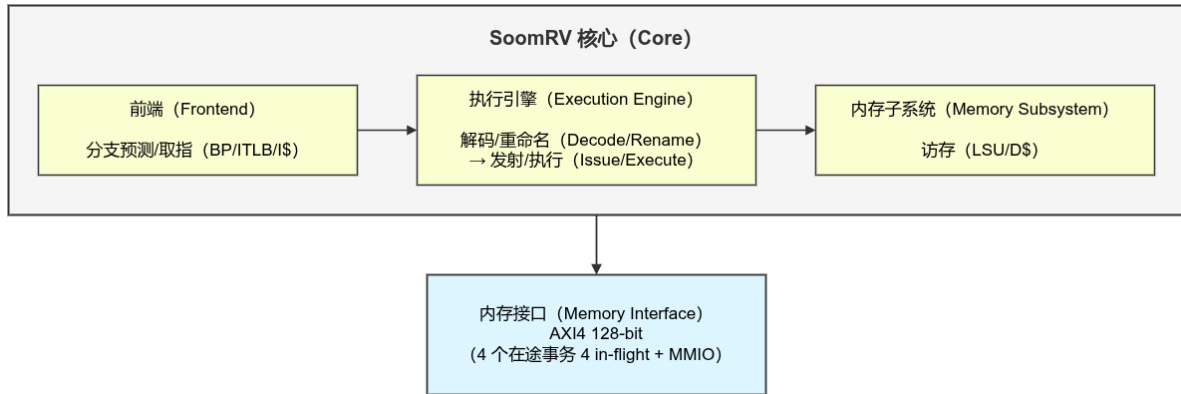


图 2-2 SoomRV 全局主干概念示意图

2.1 前端取指

前端由 `IFetch + IFetchPipeline + BranchHandler + InstrAligner` 组成，核心职责是“持续供给 + 可恢复取指”。

它不只做 ICache 读取，还负责 ITLB 翻译、cache miss 请求发起、取指包边界修正，以及将 `fetch` 元数据写入 `PCFile/BPFile` 供后端回读。

```
// src/IFetch.sv:L88-L118
BranchPredictor#(.NUM_IN(NUM_BP_UPD+1)) bp
(
    .IN_pcValid(iffetchEn),
    .OUT_fetchLimit(BP_fetchLimit), // 告诉前端哪些 fetchID 不能覆盖
    .OUT_pc(pc),                    // 下一拍取指 PC
    .OUT_predBr(predBr),           // 分支预测元数据
    .IN_retDecUpd(BH_retDecUpd),
    .IN_btUpdates({'BH_btUpdate, IN_btUpdates[1], IN_btUpdates[0]}),
    .IN_bpUpdate(IN_bpUpdate)
);

// src/IFetch.sv:L133-L179
IFetchPipeline ifp
(
    .IN_mispr(IN_branch.taken || IN_decBranch.taken),
    .IN_BP_fetchLimit(BP_fetchLimit),
    .IN_iffetchOp(iffetchOp),
    .IN_predBranch(predBr),
    .OUT_bpFileWE(BPF_we),
    .OUT_pcFileWE(pcFileWriteEn),
    .OUT_fetchBranch(BH_fetchBranch),
    .OUT_btUpdate(BH_btUpdate),
    .OUT_retUpdate(BH_retDecUpd),
    .OUT_instrs(OUT_instrs)
);

// src/IFetchPipeline.sv:L144-L155,L176-L181,L192-L201
wire FetchID_t fetchLimit = (IN_BP_fetchLimit.valid ? IN_BP_fetchLimit.fetchID :
IN_ROB_curFetchID);
if (fetchLimit == (fetchID + FetchID_t'(fetch0.valid)))
    OUT_stall = 1; // 防止覆盖仍要用于恢复/训练的 BP/PC 条目
```

```

if (IN_ifetchOp.valid && !OUT_stall) begin
    IF_icache.re = 1;
    IF_ict.re = 1;
end

TLB#(1, `ITLB_SIZE, `ITLB_ASSOC, 1) itlb
(
    .clear(IN_clearICache || IN_flushTLB),
    .IN_pw(IN_pw),
    .IN_rqs('{TLB_req}),
    .OUT_res('{TLB_res_c})
);

```

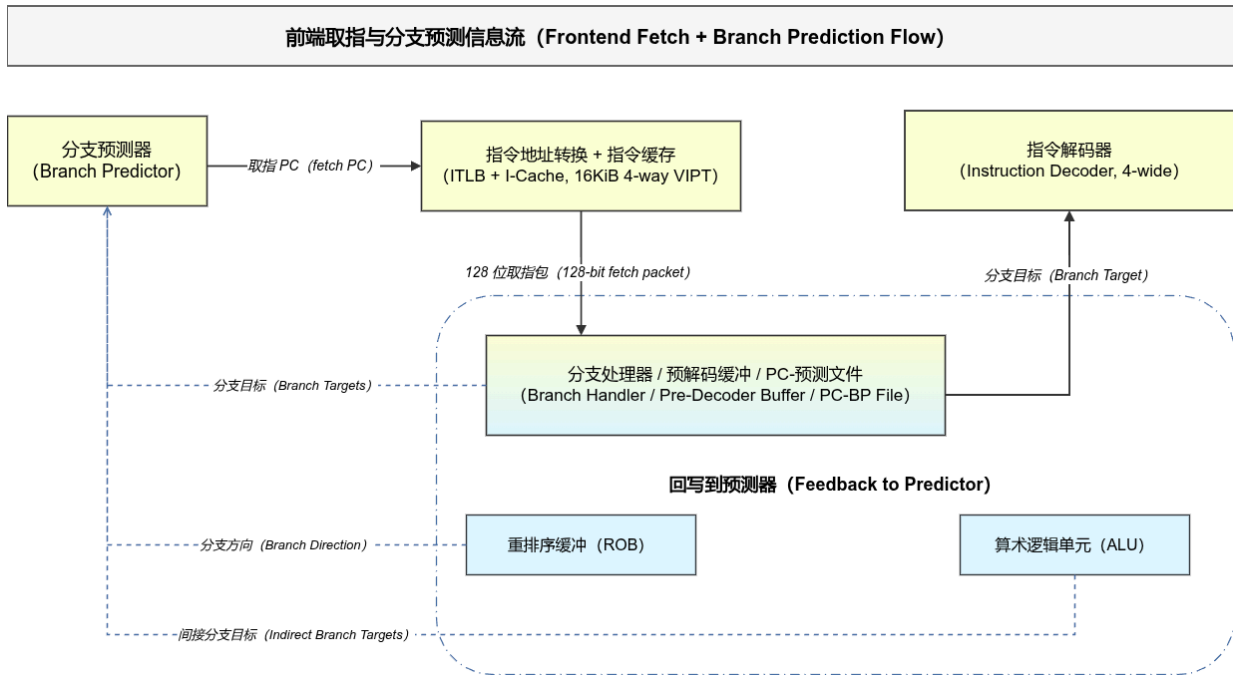


图 2-3 前端取指与分支预测闭环

2.2 分支预测

前端预测采用“目标/方向分离”：**BTB + ReturnStack** 给目标，**TAGE** 给方向；**BranchHandler** 负责在取回指令后做一次前端可判定纠偏；**ROB** 提交时再发 **BPUpdate** 训练方向，避免错误路径污染。

```

// src/BranchPredictor_bt_arb.sv:L96-L139
if (ignorePred) begin
    if (recovery.valid && recovery.tgtSpec != BR_TGT_MANUAL)
        OUT_pc = recoveredPC; // 恢复路径优先于新预测
end
else if (BTB_br.valid && BTB_br.btype != BT_RETURN) begin
    OUT_predBr = BTB_br;
    OUT_predBr.taken |= TAGE_taken; // 目标来自 BTB, 方向由 TAGE 叠加
    OUT_predBr.multiple = !OUT_predBr.taken && BTB_br.multiple;
    if (OUT_predBr.taken) OUT_pc = OUT_predBr.dst;
end
else if (BTB_br.valid && BTB_br.btype == BT_RETURN && RET_br.valid) begin
    OUT_predBr = BTB_br;
    OUT_predBr.taken = 1;
    OUT_predBr.dst = OUT_curRetAddr; // RET 目标由返回栈给出
    OUT_pc = OUT_predBr.dst;
end

```

```

end
else begin
    OUT_predBr.valid = 1;
    OUT_predBr.btype = BT_BRANCH;
    OUT_predBr.dirOnly = 1;          // 仅方向预测
    OUT_predBr.taken = TAGE_taken;
end

// src/ROB.sv:L331-L336
if (deqFlags[i] == FLAGS_PRED_TAKEN || deqFlags[i] == FLAGS_PRED_NTAKEN) begin
    OUT_bpUpdate.valid <= 1;
    OUT_bpUpdate.branchTaken <= (deqFlags[i] == FLAGS_PRED_TAKEN);
    OUT_bpUpdate.fetchID <= deqEntries[i].fetchID;
    OUT_bpUpdate.fetchOffs <= deqEntries[i].fetchOffs;
end

```

2.3 解码、重命名、发射与读操作数

这一路径把“语义展开、依赖编码、资源选择、读值”拆成独立阶段：[InstrDecoder](#) 把 ISA 指令转为内部 uOp；[Rename](#) 生成 [sqN/tag](#)；[IssueQueue](#) 在依赖满足后发射；[Load](#) 在发射后完成 RF/前递取值与 PCFile 回读。

```

// src/Core.sv:L117-L171,L180-L215,L294-L320
PD_Instr PD_instrs[`DEC_WIDTH-1:0];
D_UOp DE_uop[`DEC_WIDTH-1:0];
R_UOp RN_uop[`DEC_WIDTH-1:0];
IS_UOp IS_uop[NUM_PORTS-1:0];
EX_UOp LD_uop[NUM_PORTS-1:0];
InstrDecoder idec
(
    .en(!RN_stall && frontendEn),
    .IN_instrs(PD_instrs),
    .OUT_uop(DE_uop),
    .OUT_decBranch(decBranch)
);
Rename rn
(
    .IN_uop(DE_uop),
    .IN_comUOp(comUOps),
    .IN_branch(branch),
    .OUT_uop(RN_uop)
);
IssueQueue iq
(
    .IN_uop(RN_uop),
    .IN_flagUOp(flagUOps[NUM_PORTS-1:0]),
    .IN_branch(branch),
    .OUT_uop(IS_uop[i])
);
Load ld
(
    .IN_uop(IS_uop),
    .IN_resultUOps(resultUOps[NUM_PORTS-1:0]),
    .OUT_pcRead(PC_readReq),
    .OUT_uop(LD_uop)
);

```

```
// src/Rename.sv:L80-L117
OUT_stall = |portStall;
if (IN_mispredFlush && IN_uop[i].valid)
    OUT_stall = 1;
if ((!TB_tagsValid[i]) && IN_uop[i].valid && frontEn && TB_tagNeeded[i])
    OUT_stall = 1;

RAT_issueSqNs[i] = nextCounterSqN;
RAT_issueValid[i] = !rst && !IN_branch.taken && frontEn && !OUT_stall && IN_uop[i].valid;
if (RAT_issueValid[i])
    nextCounterSqN = nextCounterSqN + 1;

// src/IssueQueue.sv:L175-L203
deqCandidate_c[i] = (i < insertIndex) &&
    &(queueAvail_c[0][i]) &&
    (!HasFU(FU_DIV) || queue[i].fu != FU_DIV || !IN_doNotIssueDiv) &&
    (!HasFU(FU_FDIV) || queue[i].fu != FU_FDIV || !IN_doNotIssueFDiv) &&
    (!HasFU(FU_CSR) || queue[i].fu != FU_CSR || (i == 0 && queue[i].sqN == IN_commitSqN))
&&
    (!HasFU(FU_AGU) || queue[i].opcode != LSU_SC_W || (i == 0 && queue[i].sqN ==
IN_commitSqN));
```

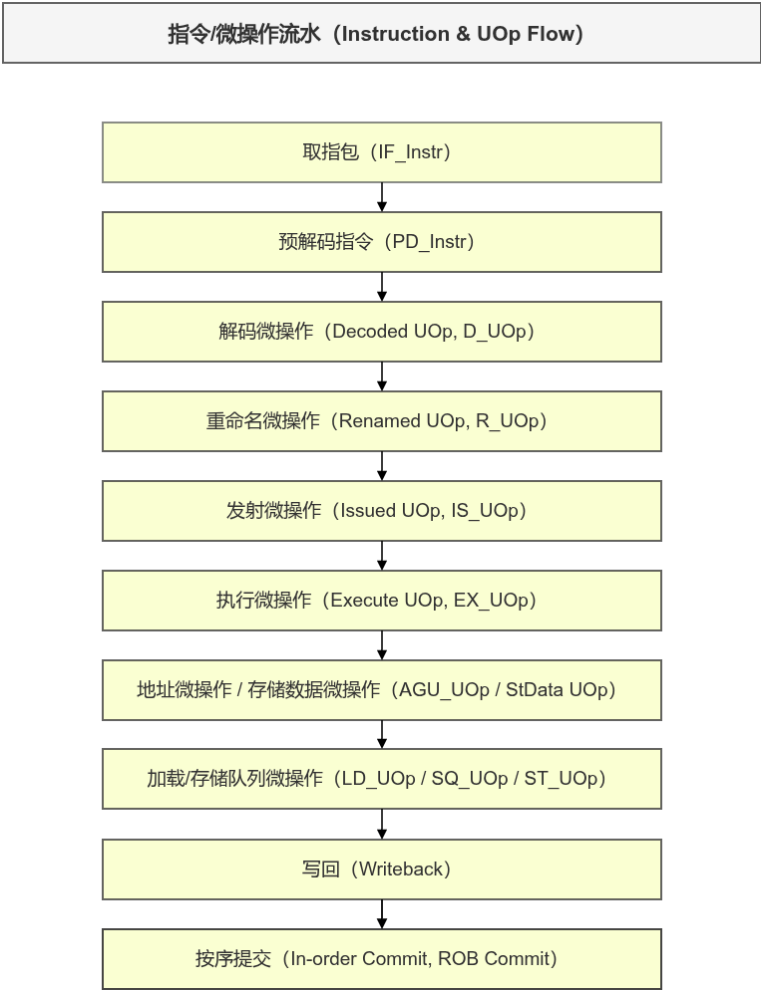


图 2-4 指令到 UOp 的主线流转

2.4 执行单元、提交与异常控制

执行阶段多单元并行（整数、乘除、浮点、AGU/LSU），但架构状态只在 ROB 按序提交时可见。

异常与顺序事件由 `TrapHandler` 收口，统一产出 flush/重定向与 CSR 陷入信息，避免多源控制流并发冲突。

```
// src/Core.sv:L351-L377,L380-L427
IntALU ialu
(
    .IN_uop(LD_uop[i]),
    .IN_branch(branch),
    .OUT_branch(ialuBranch),
    .OUT_btUpdate(BP_btUpdates[i]),
    .OUT_uop(resUOps[FU_INT])
);
Divide div(.IN_uop(LD_uop[i]), .IN_branch(branch), .OUT_uop(resUOps[FU_DIV]));
Multiply mul(.IN_uop(LD_uop[i]), .IN_branch(branch), .OUT_uop(resUOps[FU_MUL]));
FPU fpu(.IN_uop(LD_uop[i]), .IN_branch(branch), .OUT_uop(resUOps[FU_FPU]));

// src/Core.sv:L795-L827,L838-L857
ROB rob
(
    .IN_uop(RN_uop),
    .IN_flagUOps(flagUOps),
    .IN_branch(branch),
    .OUT_comUOp(comUOps),           // 按序提交
    .OUT_trapUOp(ROB_trapUOp),     // 统一异常入口
    .OUT_bpUpdate(ROB_bpUpdate),   // 提交侧方向训练
    .OUT_mispredFlush(mispredFlush)
);
TrapHandler trapHandler
(
    .IN_trapInstr(ROB_trapUOp),
    .IN_trapControl(CSR_trapControl),
    .OUT_branch(branchProvs[TH_BRANCH_PORT]),
    .OUT_flushTLB(TH_flushTLB),
    .OUT_clearICache(TH_clearICache)
);

// src/TrapHandler.sv:L101-L118,L154-L166
if (!IN_trapInstr.timeout && (
    IN_trapInstr.flags == FLAGS_FENCE ||
    IN_trapInstr.flags == FLAGS_ORDERING ||
    IN_trapInstr.flags == FLAGS_XRET
)) begin
    OUT_branch_c.taken = 1;
    OUT_branch_c.flush = 1;
    OUT_branch_c.cause = FLUSH_ORDERING;
end
```

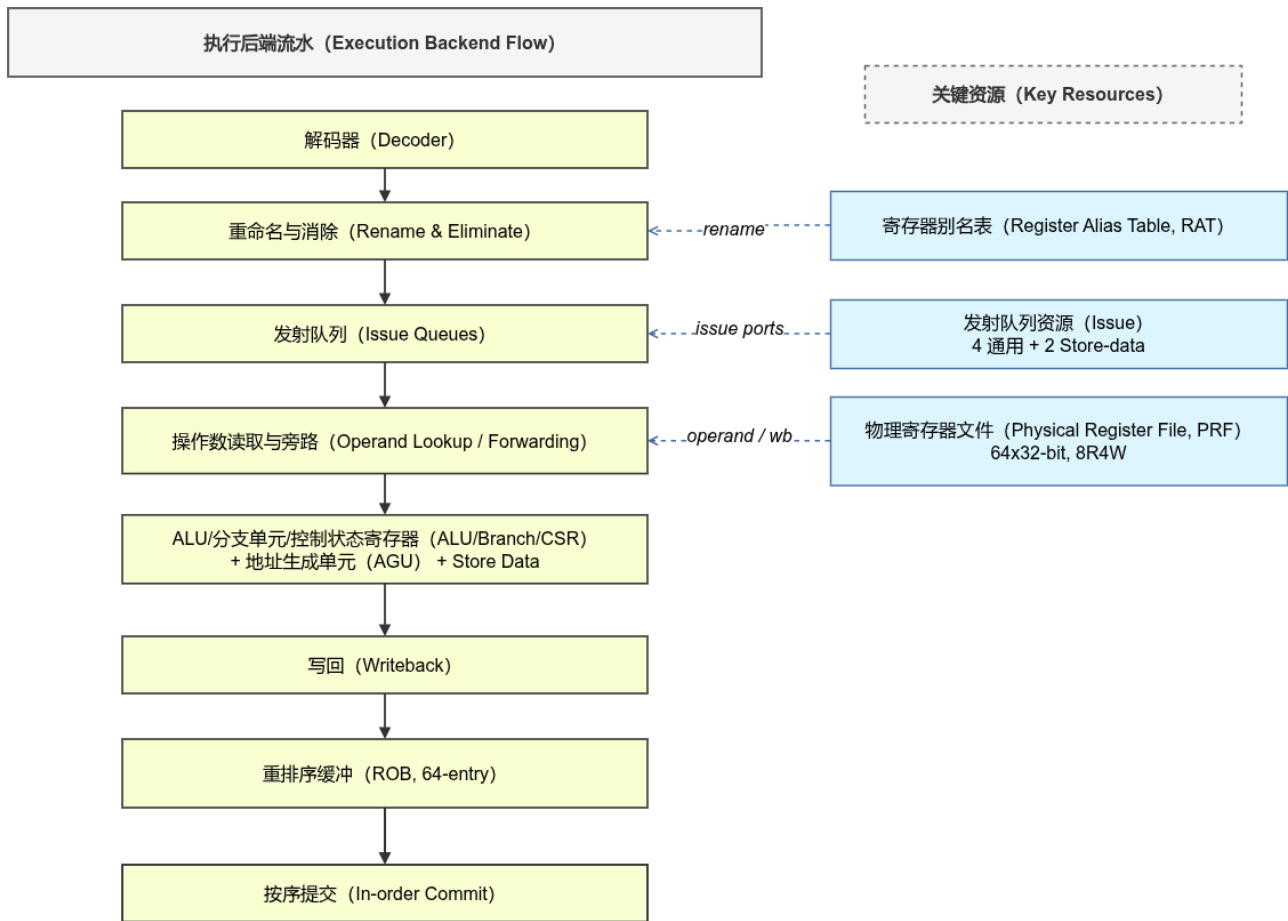



图 5 执行后端主干与关键资源关系

2.5 内存子系统与 MMU

内存链路按“快路径优先、慢路径可恢复”组织：

AGU 负责地址与异常分类；TLB miss 入 **TLBMissQueue**；**PageWalker** 做页表遍历；**LoadStoreUnit** 统一 cache/MMIO 访问；**LoadBuffer/StoreQueue** 维持内存序与回放边界。

```
// src/AGU.sv:L335-L349
if (issUOp_c.valid &&
    (!IN_branch.taken || $signed(issUOp_c.sqN - IN_branch.sqN) <= 0) &&
    (IN_vmem.sv32en && exceptFlags == FLAGS_NONE && !IN_tlb.hit)
) begin
    tlbMiss = 1;
    TMQ_enqueue = 1;
    TMQ_uopReady = 0;
end

// src/AGU.sv:L367-L381
if (pageWalkActive) begin
    if (!pageWalkAccepted) begin
        if (IN_pw.busy && IN_pw.rqID == $bits(IN_pw.rqID)'(RQ_ID))
            pageWalkAccepted <= 1;
        else begin
            OUT_pw.valid <= 1;
            OUT_pw.rootPPN <= IN_vmem.rootPPN;
            OUT_pw.addr <= pageWalkAddr;
        end
    end
end
```

```

        else if (IN_pw.valid) begin
            pageWalkActive <= 0;
            pageWalkAccepted <= 0;
        end
    end
end

// src/TLBMissQueue_param.sv:L79-L89
if (IN_pw.valid) begin
    for (integer i = 0; i < SIZE; i=i+1) begin
        if (queue[i].valid && !ready[i] &&
            (IN_pw.isSuperPage ? (IN_pw.vpn[19:10] == queue[i].addr[31:22]) : (IN_pw.vpn ==
queue[i].addr[31:12])))
            ) begin
                ready[i] <= 1;
            end
        end
    end
end

// src/PageWalker_pw_arb.sv:L30-L38,L118-L139
PageWalkReqArbiter#(.NUM_RQS(NUM_RQS)) reqArb
(
    .IN_valid(rqValid),
    .IN_start(rrStart),
    .OUT_idx(selRqIdx),
    .OUT_valid(selRqValid)
);
if (selRqValid) begin
    OUT_ldUOp.addr <= {IN_rqs[selRqIdx].rootPPN[19:0], IN_rqs[selRqIdx].addr[31:22], 2'b0};
    if (selRqIdx == RqIdx_t'(NUM_RQS - 1)) rrStart <= '0; // round-robin
    else rrStart <= selRqIdx + RqIdx_t'(1);
end
end

```

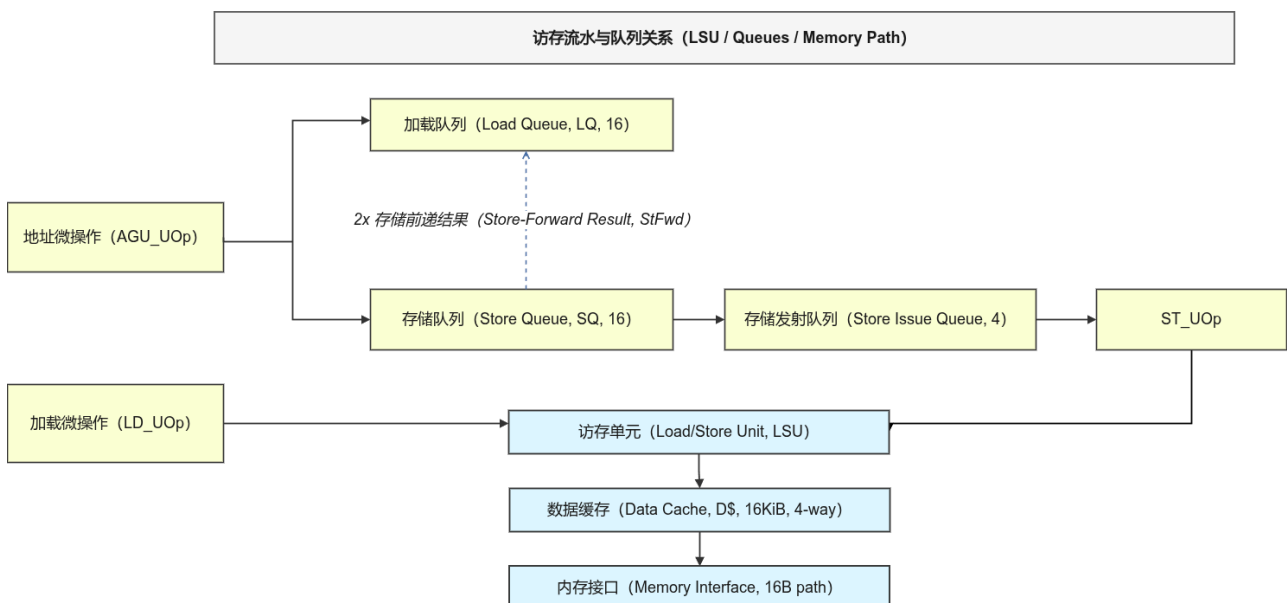


图6 访存路径、LQ/SQ 与 LSU 关系

2.6 全局串联与系统集成

系统层遵循“Top 最小化、SoC 汇聚、MemoryController 事务化”的分工：

Top 只连接 SoC 与外部 AXI 仿真；**SoC** 组织核内 cache/memc 接口；**MemoryController** 负责 cache line 事务与冲突仲裁。

```
// src/Top.sv:L11-L13,L47-L50,L85-L90
assign OUT_halt = SOC_poweroff || SOC_reboot;
ExternalAXISim extMem
(
    .clk(clk),
    .rst(rst),
    .s_axi_awid(s_axi_awid),
    .s_axi_arid(s_axi_arid),
    .s_axi_rdata(s_axi_rdata)
);
SoC soc
(
    .clk(clk),
    .rst(rst),
    .en(en),
    .OUT_powerOff(SOC_poweroff),
    .OUT_reboot(SOC_reboot)
);

// src/SoC.sv:L64-L73,L124-L143
MemoryController memc
(
    .IN_ctrl(MemC_ctrl),
    .OUT_stat(MemC_stat),
    .OUT_icacheW(MC_IC_wr),
    .OUT_dcacheW(MC_DC_wr)
);
Core core
(
    .OUT_memc(MemC_ctrl),
    .IN_memc(MemC_stat),
    .IF_cache(IF_cache),
    .IF_icache(IF_icache)
); // SoC 内通过 memc 接口解耦 core 与 AXI

// src/MemoryController.sv:L137-L157
if (!cacheAddrColl) begin
    selReq = IN_ctrl[i];
    OUT_stat.stall = ~(1 << i); // 仅放行一个无冲突请求
end
```

3 详细实现

3.1 分支预测实现

3.1.1 BTB/RET/TAGE 各自解决什么问题

在 SoomRV 的实现里，[目标预测](#) 与 [方向预测](#) 是两条独立路径，这个拆分直接对应不同统计特征与硬件约束。

[BTB](#) (Branch Target Buffer) 解决的是“如果要跳，跳到哪里”。它本质是一个按 PC 索引的目标缓存，返回 [btype + dst + offs](#)。与教材里常见的“只做方向、不预测目标”的 1-bit/2-bit BHT 相比，BTB 的直接收益是把 **taken** 分支的重定向从“执行后”前移到“取指前”，减少 **taken** 分支的固定气泡。

[RET](#) 预测 (Return Address Stack) 专门覆盖函数返回。把返回目标塞进普通 BTB 也能工作，但在存在大量 **call-site** 复用时，返回目标容易被别名污染。返回栈的优势是把“调用-返回”关系建模为栈语义，命中与程序结构一致；代价是必须处理回滚与恢复，这也是 [ReturnStack](#) 代码复杂度高于 BTB 的原因。

[TAGE](#) (TAGged GEometric history length) 解决的是“要不要跳”。教材中常见的 bimodal (2-bit 饱和计数器) 对短局部模式有效，但遇到跨基本块相关性时容易失效；gshare 通过全局历史异或 PC 缓解了一部分问题，但不同历史长度模式会竞争同一表项。TAGE 通过“多历史长度分层 + tag 校验 + altPred 回退”把这些模式拆开建模，通常能在同等容量下得到更稳的方向准确率。

SoomRV 的组合策略可以概括为：[BTB/RET](#) 负责“尽早给目标”，[TAGE](#) 负责“尽量给对方向”，[ROB](#) 负责“只在提交时训练方向”。这样做比经典“执行级直接更新 BHT”更保守，但在长程序与复杂控制流上更抗错误路径污染。

3.1.2 设计拆分：目标与方向解耦

在前端视角里，分支预测本质是两个问题：

1. [是否改向](#) (direction)：下一拍是否离开顺序 PC。
2. [改向到哪里](#) (target)：若改向，目标 PC 是什么。

这两个问题的统计特征并不一致。方向通常依赖历史模式（循环、相关分支链），目标通常依赖局部地址上下文（直接分支目标）或调用栈上下文（返回地址）。因此 SoomRV 没有使用单一预测器一把抓，而是做了明确拆分：

- [BTB](#)：提供大多数直接分支/跳转的候选目标与分支类型。
- [ReturnStack](#)：专门处理 [RET](#) 目标，避免返回目标被 BTB 别名污染。
- [TAGE](#)：只负责方向，给出 **taken/not-taken**。

最终在 [BranchPredictor](#) 内按优先级融合：先恢复，再尝试目标预测 (BTB/RET)，最后退化到只给方向的 [dirOnly](#)。

```
// src/BranchPredictor_bt_arb.sv:L96-L140
always_comb begin
    OUT_predBr = '0;
    OUT_pc = pcReg;

    if (ignorePred) begin
        if (recovery.valid && recovery.tgtSpec != BR_TGT_MANUAL)
            OUT_pc = recoveredPC;
        end
    else if (BTB_br.valid && BTB_br.btype != BT_RETURN) begin
        // 目标来自 BTB，方向由 TAGE 叠加
        OUT_predBr = BTB_br;
        OUT_predBr.taken |= TAGE_taken;
        if (OUT_predBr.taken) OUT_pc = OUT_predBr.dst;
        end
    else if (BTB_br.valid && BTB_br.btype == BT_RETURN && RET_br.valid) begin
        // RET 目标来自返回栈
```

```

        OUT_predBr.taken = 1;
        OUT_predBr.dst = OUT_curRetAddr;
        OUT_pc = OUT_predBr.dst;
    end
    else begin
        // 无目标命中时仍输出方向预测 (dirOnly)
        OUT_predBr.valid = 1;
        OUT_predBr.btype = BT_BRANCH;
        OUT_predBr.dirOnly = 1;
        OUT_predBr.taken = TAGE_taken;
    end
end
end

```

这套拆分的核心优势不是“单点命中率更高”，而是可恢复性与可训练性更稳定：

1. 目标错误与方向错误可以分开诊断，避免调参时相互掩盖。
2. 目标路径可由 `BTUpdate` 在前端/执行侧尽早修补；方向路径由 `BPUUpdate` 在提交侧训练，错误路径不会污染 TAGE。
3. 当目标缺失时保留 `dirOnly`，后端仍可利用方向信息，不会退化成完全无预测。

代价同样明确：前端融合逻辑更复杂，需要额外状态保护（如 `fetchLimit` 防止 PC/BP 文件被提前覆盖），并在高压场景下引入一定节流。该代价换来的是长期运行下更可控的误判行为，尤其是在 Linux 这类长程序负载中更明显。

3.1.3 BTB：直接映射 + 标签到位检查

BTB 采用直接映射数组，先读后比 tag，保证可综合为同步 RAM：

```

// src/BranchTargetBuffer.sv:L40-L65
always_ff@(posedge clk) begin
    if (IN_pcValid) begin
        fetched.entry <= entries[IN_pc[$clog2(LENGTH)-1:0]];
        fetched.multiple <= multiple[IN_pc[$clog2(LENGTH)-1:0]];
        fetched.pc <= IN_pc;
    end
end

always_comb begin
    OUT_branch = PredBranch'{valid: 0, default: 'x};
    if (fetched.entry.valid &&
        fetched.entry.src == fetched.pc[$clog2(LENGTH)+: `BTB_TAG_SIZE] &&
        fetched.entry.offsets >= fetched.pc[0+: $bits(FetchOff_t)]) begin
        OUT_branch.valid = 1;
        OUT_branch.dst = fetched.entry.dst;
        OUT_branch.btype = fetched.entry.btype;
        OUT_branch.offsets = fetched.entry.offsets;
        OUT_branch.taken = fetched.entry.btype == BT_CALL || fetched.entry.btype ==
BT_JUMP;
    end
end

```

更新路径里有个容易被忽略的细节：`multiple` 位于同一 fetch 包存在多个分支时的截断控制，避免“第一个不跳转但后面其实有分支”导致前端跨包取指。

3.1.4 BT 更新仲裁：避免同拍丢更新

多执行端口会同拍产生多个 `BTUpdate`。默认实现通过 `BTUpdateArbiter` 做显式仲裁与缓存：

```
// src/BTUpdateArbiter.sv:L34-L71
// 1) 先服务 pending，避免历史更新饿死
for (integer i = NUM_IN - 1; i >= 0; i=i-1)
    if (!selValid_c && pendingValid_r[i]) begin
        selValid_c = 1;
        selFromPending_c = 1;
        OUT_update = pending_r[i];
    end

// 2) 再选当拍输入（高索引优先）
for (integer i = NUM_IN - 1; i >= 0; i=i-1)
    if (!selValid_c && IN_updates[i].valid) begin
        selValid_c = 1;
        selFromPending_c = 0;
        OUT_update = IN_updates[i];
    end

// 3) 未被消费的当拍输入写入 per-source pending
if (IN_updates[i].valid && !consumedNow && !pendingValid_c[i]) begin
    pending_c[i] = IN_updates[i];
    pendingValid_c[i] = 1;
end
```

相比“循环覆盖最后一条有效更新”的简化写法，这个版本解决了并发更新丢失问题，代价是每源一个 pending 槽位和轻微时序压力。

3.1.5 RET 预测：可恢复返回栈

`ReturnStack` 不只是 push/pop，它还维护了用于回滚的覆盖日志队列 `rrqueue`：

```
// src/ReturnStack.sv:L62-L73,L194-L200
always_comb begin
    rindex = rindexReg;
    if (forwardRindex)
        rindex = IN_recoveryIdx; // mispredict 当拍前递恢复索引
    else if (IN_branch.valid && IN_branch.btype == BT_CALL && lastValid)
        rindex = rindex + 1;
    else if (IN_branch.valid && IN_branch.btype == BT_RETURN && lastValid)
        rindex = rindex - 1;
end

// 恢复期：把 rrqueue 中的旧值回写到 rstack
if (recoveryInProgress && recoveryContinue_c) begin
    rstack[rrqueue[qindex-1].idx] <= rrqueue[qindex-1].addr;
    qindex_r <= qindex_r - 1;
end
```

实现痛点在于 fetchID 不是天然可比较有序（无额外环绕位），代码里采用“相对基准 fetchID”的比较来决定回收和恢复顺序，这也是返回栈恢复复杂度高于 BTB 的主要原因。

3.1.6 TAGE 方向预测：分层命中、按需分配、Useful 衰减

TAGE 由 base bimodal + 多级 tagged table 组成。核心输出逻辑是“命中最深表覆盖浅层结果，同时保存 altPred”：

```
// src/TagePredictor.sv:L154-L165
always_comb begin
    OUT_altPred = predictions[0];
    OUT_predTaken = predictions[0];
    OUT_predTageID = 0;

    for (integer i = 0; i < NUM_STAGES; i=i+1) begin
        if (valid[i]) begin
            OUT_predTageID = i[$bits(TageID_t)-1:0];
            OUT_altPred = OUT_predTaken;
            OUT_predTaken = predictions[i];
        end
    end
end
```

地址与历史混合索引采用 XOR folding：

```
// src/TagePredictor.sv:L69-L80
for (integer j = 0; j < ($bits(IN_predAddr)/HASH_SIZE); j=j+1)
    predHashes[i] = predHashes[i] ^ IN_predAddr[j*HASH_SIZE+:HASH_SIZE];

for (integer j = 0; j < (BASE * (FACTOR ** i)); j=j+1) begin
    predHashes[i][j % HASH_SIZE] ^= IN_predHistory[j];
    predTags[i][j % TAG_SIZE] ^= IN_predHistory[j] ^ IN_predHistory[(j+1) % hist_bits];
end
```

表项更新由 `TageTable` 管理 useful 与分配：

```
// src/TageTable.sv:L87-L105
if (IN_writeUpdate) begin
    if (IN_writeUseful) begin
        if (IN_writeCorrect && useful[IN_writeAddr] != {USF_SIZE{1'b1}})
            useful[IN_writeAddr] <= useful[IN_writeAddr] + 1;
        else if (!IN_writeCorrect && useful[IN_writeAddr] != {USF_SIZE{1'b0}})
            useful[IN_writeAddr] <= useful[IN_writeAddr] - 1;
    end
end
else if (IN_doAlloc) begin
    if (useful[IN_writeAddr] == 0)
        tag[IN_writeAddr] <= IN_writeTag;
end
```

此外，默认 `BHT_IMPL=bht_fwd` 解决了 base BHT 同索引连续训练的写后读滞后：

```
// src/BranchPredictionTable_bht_fwd.sv:L74-L77
// 同拍写回结果前递给本拍采样，避免 stale read-after-write
if (write_c.valid && write_r.valid && write_r.addr == write_c.addr)
    writeTempFwd = writeCounterNext;
```

3.1.7 恢复与训练防污染：提交侧方向更新

方向更新并不在执行时立即写入预测器，而是由 ROB 在提交时发出 `BPUpdate`：

```
// src/ROB.sv:L331-L337
if (deqFlags[i] == FLAGS_PRED_TAKEN || deqFlags[i] == FLAGS_PRED_NTAKEN) begin
    OUT_bpUpdate.valid <= 1;
    OUT_bpUpdate.branchTaken <= (deqFlags[i] == FLAGS_PRED_TAKEN);
    OUT_bpUpdate.fetchID <= deqEntries[i].fetchID;
    OUT_bpUpdate.fetchOffs <= deqEntries[i].fetchOffs;
end
```

前端通过 `fetchLimit` 与更新 FIFO 保护尚未读取的 BP/PC 文件项，避免“训练需要读旧状态，但条目已被新取指覆盖”：

```
// src/BranchPredictor_bt_arb.sv:L275-L285
if (bpUpdate.valid) begin
    OUT_fetchLimit.valid = 1;
    OUT_fetchLimit.fetchID = bpUpdate.fetchID;
end
else if (IN_bpUpdate.valid) begin
    OUT_fetchLimit.valid = 1;
    OUT_fetchLimit.fetchID = IN_bpUpdate.fetchID;
end
```

这一策略增加了少量前端节流，但换来了稳定的训练一致性。在 Linux 这类长程序里，这个权衡通常是值得的。

3.1.8 BranchHandler 的前端纠偏职责

预测器并不是前端唯一的“分支模块”。`BranchHandler` 在 ICache 返回后会做一次轻量级语义校验，负责修正“同包内能确定答案”的场景：例如直接跳转目标、非法预测位置、未预测到的静态可判定跳转。

```
// src/BranchHandler.sv:L306-L438
// 预测命中但目标/类型不合法：触发清理并重抓
btUpdate_c.valid = 1;
btUpdate_c.clean = 1;
btUpdate_c.src = {IN_op.pc[31:$bits(FetchOff_t)], IN_op.predBr.offs};

decBranch_c.taken = 1;
decBranch_c.fetchID = IN_op.fetchID;
decBranch_c.fetchOffs = FetchOff_t'(i);
decBranch_c.tgtSpec = BR_TGT_MANUAL;

// 未预测到但本包内发现直接分支：登记 BTB 目标
if (!predicted &&
    (curBr.btype == JUMP || curBr.btype == CALL || curBr.btype == BRANCH || curBr.btype ==
    RETURN)) begin
    btUpdate_c.valid = 1;
    btUpdate_c.clean = 0;
    btUpdate_c.btype = curBr.btypeSimple;
    btUpdate_c.src = curBr.fhPC;
    btUpdate_c.dst = curBr.target;
end
```

这个模块的价值在于把“可在 fetch 后立刻判定的错误”尽量前移修正，把执行级留给真正需要寄存器值才能判定的分支方向和间接目标。这样既减小了错误路径深度，也避免了后端承担不必要的前端别名成本。

3.2 发射乱序执行实现

3.2.1 4 宽入口与 5 端口执行资源

四发射在 SoomRV 中指“每拍最多 4 条新指令进入 OoO 机器并按 4 宽提交”，不是“只有 4 个执行端口”。

```
// src/Core.sv:L117-L123
PD_Instr PD_instrs[`DEC_WIDTH-1:0]; // 4 路前端输出
D_UOp DE_uop[`DEC_WIDTH-1:0];
R_UOp RN_uop[`DEC_WIDTH-1:0];
CommitUOp comUOps[`DEC_WIDTH-1:0]; // 4 路提交

IS_UOp IS_uop[NUM_PORTS-1:0]; // NUM_PORTS = 5
EX_UOp LD_uop[NUM_PORTS-1:0];
```

因此系统可在队列有积压时出现“同拍执行 uOp 数 > 新进入指令数”，但架构提交吞吐上限仍由 `DEC_WIDTH=4` 决定。

3.2.2 Rename：标签分配与三类序号协同

Rename 同时处理三件事：

- 架构寄存器到物理标签映射（RAT + TagBuffer）。
- 全局顺序号 `sqN` 分配。
- 访存顺序号 `loadSqN/storeSqN` 分离分配。

```
// src/Rename.sv:L80-L110,L325-L353
OUT_stall = |portStall;
if (IN_mispredFlush && IN_uop[i].valid)
    OUT_stall = 1;
if ((!TB_tagsValid[i]) && IN_uop[i].valid && frontEn && TB_tagNeeded[i])
    OUT_stall = 1;
```

```
RAT_issueValid[i] = !rst && !IN_branch.taken && frontEn && !OUT_stall && IN_uop[i].valid;
RAT_issueSqNs[i] = nextCounterSqN;
```

```
OUT_uop[i] <= R_UOp'{
    availA:    RAT_lookupAvail[2*i+0],
    tagA:      RAT_lookupSpecTag[2*i+0],
    availB:    RAT_lookupAvail[2*i+1],
    tagB:      RAT_lookupSpecTag[2*i+1],
    sqN:       RAT_issueSqNs[i],
    tagDst:    newTags[i],
    fetchID:   IN_uop[i].fetchID,
    fetchOffs: IN_uop[i].fetchOffs,
    storeSqN:  storeSqNs[i+1], // store sqn pre-increment
    loadSqN:   loadSqNs[i],    // load sqn post-increment
    valid:     1'b1,
    validIQ:   {NUM_PORTS_TOTAL{1'b1}},
    default:   'x
};
```

`storeSqN` 与 `loadSqN` 采用不同增量语义的原因，是为了让 LQ/SQ 窗口边界在 issue 阶段能直接比较，减少后级推断逻辑。

3.2.3 调度与发射：静态能力约束 + 动态就绪约束

Scheduler 负责把同拍进入的 uOp 分配到可执行端口，并在端口间做轮转优先级：

```
// src/Scheduler_param.sv:L18-L37
function automatic Candidates_t GetCandidates(FuncUnit fu, SqN sqN, SqN loadSqN, SqN
storeSqN);
    case (fu)
        FU_ATOMIC: retval = 1 << (storeSqN % NUM_AGUS); // 原子按 storeSqN 分流到 AGU 侧
        default: begin
            for (integer i = 0; i < NUM_ALUS; i=i+1)
                if (PORT_FUS[i][fu] != 0) retval[i] = 1'b1;
            end
        endcase
    endfunction
```

IssueQueue 则执行“能不能发”的最终判定：

```
// src/IssueQueue.sv:L178-L203
deqCandidate_c[i] = (i < insertIndex) &&
    &(queueAvail_c[0][i]) &&
    (!HasFU(FU_DIV) || queue[i].fu != FU_DIV || !IN_doNotIssueDiv) &&
    (!HasFU(FU_FDIV) || queue[i].fu != FU_FDIV || !IN_doNotIssueFDiv) &&

    // CSR 与 SC 保序
    (!HasFU(FU_CSR) || queue[i].fu != FU_CSR || (i == 0 && queue[i].sqN == IN_commitSqN))
&&
    (!HasFU(FU_AGU) || queue[i].opcode != LSU_SC_W || (i == 0 && queue[i].sqN ==
IN_commitSqN)) &&

    // LQ/SQ 容量边界
    (!HasFU(FU_AGU) || queue[i].opcode >= LSU_SC_W || $signed(queue[i].loadSqN -
IN_maxLoadSqN) <= 0) &&
    (!HasFU(FU_AGU) || queue[i].opcode < LSU_SC_W || $signed(queue[i].storeSqN -
IN_maxStoreSqN) <= 0);
```

这个判定式很长，但它把“正确性条件”留在发射前，避免在执行后做大范围回滚。

3.2.4 Load：发射后读操作数与多源前递

SoomRV 采用“先发射 uOp，再在 Load 阶段读值”的组织方式：

```
// src/Load.sv:L44-L47,L92-L94,L162-L170,L117-L121
// 1) 组合前递网络：ZC + WB
forwards[i+NUM_ZC_FWDS].valid = IN_resultUOps[i].valid &&
!IN_resultUOps[i].tagDst[$bits(Tag)-1];

// 2) 若 tag 命中前递则直取，否则发起 RF 读
if (IN_uop[i].tagA[$bits(Tag)-1])
    outUOpReg[i].srcA <= {{26{IN_uop[i].tagA[5]}}, IN_uop[i].tagA[5:0]};
else if (matchValid[i])
    outUOpReg[i].srcA <= forwards[matchIdx[i]].result;
else
    operandIsReg[i][0] <= 1;

// 3) 读取 PCFile，把 fetch 期元数据补回执行 uOp
OUT_pcRead[i].addr = IN_uop[i].fetchID;
```

```
OUT_uop[i].pc = {IN_pcReadData[i].pc[30:$bits(FetchOff_t)], outUOpReg[i].fetchOffs, 1'b0};
```

这一路径的收益是压短 issue 关键路径，代价是 Load 级逻辑更重。对 4 宽机器而言，这个权衡比“在 IQ 内保存完整操作数”更容易控面积和时序。

3.2.5 执行并行、按序提交与异常收敛

执行面并行写回后，ROB 统一做提交资格判定：

```
// src/ROB.sv:L282-L293,L309-L333
reg isRenamed = (i[$clog2(LENGTH):0] < $signed(lastIndex - baseIndex));
reg isExecuted = deqFlags[i] != FLAGS_NX;
reg noFlagConflict = (!pred || (deqFlags[i] == FLAGS_NONE));
reg lbAllowsCommit = (!IN_ldComLimit.valid || $signed(loadSqN - IN_ldComLimit.sqN) < 0);
reg sqAllowsCommit = 1;
for (integer j = 0; j < NUM_AGUS; j=j+1)
    sqAllowsCommit &= (!IN_stComLimit[j].valid || $signed(storeSqN - IN_stComLimit[j].sqN)
< 0);
SqN sqN = GetSqN(id);

if (!temp && isRenamed &&
    ((isExecuted && noFlagConflict && sqAllowsCommit && lbAllowsCommit) || timeoutCommit))
begin
    OUT_comUOp[i].rd <= deqEntries[i].rd;
    OUT_comUOp[i].tagDst <= deqEntries[i].tag;
    OUT_comUOp[i].sqN <= sqN;
    OUT_comUOp[i].valid <= 1;
end
```

分支误判后的 Rename 恢复采用“回到 committed 映射 + ROB 回放未提交旧指令”的方式：

```
// src/RenameTable.sv:L84-L90
if (IN_mispred) begin
    for (integer i = 1; i < NUM_REGS; i=i+1)
        specTag[i] <= comTag[i];
end

// src/ROB.sv:L165-L173
if (IN_branch.taken) begin
    misprReplay_c = MisprReplay'{
        endSqN: IN_branch.sqN,
        iterSqN: baseIndex,
        valid: 1
    };
end
```

相比每拍快照 RAT，这种方案硬件代价更低；代价是误判后需要几个周期 replay，恢复延迟取决于分支与提交指针距离。

3.2.6 4 发射方案的实现权衡

这个 OoO 实现的取舍可以概括为三点：

1. 用标签而非物理寄存器索引在 IQ 内追踪依赖，缩小队列项宽度。
2. 把“复杂但必须保序”的条件（CSR/SC、LQ/SQ 边界）留在发射判定，减少后级异常路径复杂度。
3. 提交侧统一生成架构可见事件（包括方向训练），牺牲一拍学习时效，换长期稳定性。

4 优化与改进

优化与改进的目标是解决已经暴露的稳定性问题、计数口径问题和验证链路问题。
从实现层面看，改动集中在三个方向：

1. 消除同拍并发与慢路径上的行为歧义，降低随机性。
2. 把隐式约束改为显式协议，提升可验证性与可解释性。
3. 统一构建与回归入口，让 A/B 对照、回滚和复现都可脚本化执行。

4.1 开关化组织与实现边界控制

在复杂流水线工程里，直接覆盖原实现短期更快，但长期风险更高：一旦出现回退，很难区分是“思路有问题”还是“细节实现偏差”。因此本轮统一采用“原实现保留 + 优化实现并存 + 顶层开关选择”的组织方式。

```
# Makefile:L16-L107
TLB_IMPL ?= fixed_sp_dedup
BRANCH_PRED_IMPL ?= bt_arb
BHT_IMPL ?= bht_fwd
PAGEWALKER_IMPL ?= pw_arb
TLBMISSQ_IMPL ?= tmq_param
HARDCODE4_IMPL ?= param
LS_ISSUE_IMPL ?= issue_opt

ifeq ($(BRANCH_PRED_IMPL),orig)
BP_SRC := src/BranchPredictor.sv
else ifeq ($(BRANCH_PRED_IMPL),bt_arb)
BP_SRC := src/BranchPredictor_bt_arb.sv
endif
```

这个结构有三个工程收益：

1. 对比口径稳定：同一仓库同一套测试可直接做 `orig/opt` 对照。
2. 回滚成本低：不需要临时补丁，切换开关即可回退。
3. 风险隔离清晰：每项优化的影响边界与验证入口一一对应。

代价同样明确：代码会有阶段性重复，构建配置更复杂。权衡后优先选择“可验证性”，因为该项目当前阶段的主要风险不在“写不出新逻辑”，而在“难以证明改动真的更好”。

4.2 前端预测链路收敛

4.2.1 BT 更新仲裁：从覆盖语义改为显式协议

分支目标更新最初是“同拍最后写入生效”的隐式覆盖语义。它在低并发场景能工作，但在同拍多源有效时会丢失更新，并且行为依赖输入排列顺序。

当前实现将仲裁抽出为独立模块，采用“pending 优先 + 当拍选择 + 每源单槽缓存”的协议。

```
// src/BTUpdateArbiter.sv:L32-L71
for (integer i = NUM_IN - 1; i >= 0; i=i-1)
    if (!selValid_c && pendingValid_r[i]) begin
        selValid_c = 1;
        selFromPending_c = 1;
        OUT_update = pending_r[i];
    end
```

```

for (integer i = NUM_IN - 1; i >= 0; i=i-1)
  if (!selValid_c && IN_updates[i].valid) begin
    selValid_c = 1;
    selFromPending_c = 0;
    OUT_update = IN_updates[i];
  end

  if (IN_updates[i].valid && !consumedNow && !pendingValid_c[i]) begin
    pending_c[i] = IN_updates[i];
    pendingValid_c[i] = 1;
  end
end

```

与此同时，预测器在提交侧方向更新期间给前端下发 `fetchLimit`，避免更新所需的 BP/PC 条目被新取指提前覆盖。

```

// src/BranchPredictor_bt_arb.sv:L275-L285
OUT_fetchLimit = FetchLimit'{valid: 0, default: 'x};
if (bpUpdate.valid) begin
  OUT_fetchLimit.valid = 1;
  OUT_fetchLimit.fetchID = bpUpdate.fetchID;
end
else if (IN_bpUpdate.valid) begin
  OUT_fetchLimit.valid = 1;
  OUT_fetchLimit.fetchID = IN_bpUpdate.fetchID;
end
end

```

这里刻意没有直接上全局 FIFO，而是先用“每源单槽 pending”覆盖核心故障模式。该方案在逻辑复杂度、时序压力和可靠性之间更均衡，后续是否扩展 FIFO 取决于压力样本，而非预设复杂化。

4.2.2 BHT 写后读前递：修复连续同索引训练滞后

基础 BHT 训练是两拍链路：本拍读旧计数器，下拍写回新计数器。若同索引连续训练，读口可能拿到 stale 值，导致方向学习滞后。

优化版本加入同拍 RAW 前递，不改变模块接口与时序边界。

```

// src/BranchPredictionTable_bht_fwd.sv:L65-L77
writeCounterNow = {pred[write_c.addr], hist[write_c.addr]};
writeCounterNext = nextCounter(writeTempReg, write_r.taken, write_r.init);
writeTempFwd = writeCounterNow;

// Forward same-cycle writeback result to avoid stale read-after-write.
if (write_c.valid && write_r.valid && write_r.addr == write_c.addr)
  writeTempFwd = writeCounterNext;

```

这个修复看似局部，但它直接影响方向预测收敛速度。相比重写整个 BHT 结构，这种“局部前递”更符合本阶段目标：最小侵入地消除可复现错误行为。

4.3 Load/Store 发射节拍修复

Store 发射口在持续 backpressure 下曾出现 `valid` 抖动，表现为 `1/0/1/0` 周期性空泡。问题本质是 ready/valid 语义没有完整实现“未被接受即保持稳定”的约束。

优化后的策略是“仅在被接受时清空，否则保持输出稳定”。

```
// src/StoreQueueBackend_issue_opt.sv:L214-L247
issuePortFree = !OUT_uopSt.valid || !IN_stallSt;

if (OUT_uopSt.valid && !IN_stallSt)
    OUT_uopSt <= ST_UOp'{valid: 0, default: 'x};

if (issuePortFree && reIssue.valid) begin
    evicted[reIssue.idx].issued <= 1;
    OUT_uopSt.valid <= 1;
    OUT_uopSt.id <= reIssue.idx;
    OUT_uopSt.nonce <= evicted[reIssue.idx].nonce;
end
```

这个改法没有引入新握手信号，也没有改变上下游协议，只校正了状态机在阻塞条件下的行为，从而减少无意义发射空泡。

4.4 MMU 慢路径与参数化一致性

4.4.1 PageWalker 请求仲裁显式化

PageWalker 请求选择由“循环覆盖”改为独立仲裁模块，策略是轮转起点扫描（round-robin start）。

```
// src/PageWalkReqArbiter.sv:L15-L27
for (integer offs = 0; offs < NUM_RQS; offs=offs+1) begin
    integer cand;
    cand = IN_start + offs;
    if (cand >= NUM_RQS) cand = cand - NUM_RQS;
    if (!OUT_valid && IN_valid[cand]) begin
        OUT_idx = RqIdx_t'(cand);
        OUT_valid = 1;
    end
end
```

主状态机只保留“用仲裁结果发起页表访问”和“推进 `rrStart`”两件事。

```
// src/PageWalker_pw_arb.sv:L118-L139
if (selRqValid) begin
    OUT_ldUOp.addr <= {IN_rqs[selRqIdx].rootPPN[19:0], IN_rqs[selRqIdx].addr[31:22], 2'b0};
    if (selRqIdx == RqIdx_t'(NUM_RQS - 1)) rrStart <= '0;
    else rrStart <= selRqIdx + RqIdx_t'(1);
end
```

这种拆分让仲裁公平性和页表遍历逻辑解耦，验证粒度更细，排障路径更短。

4.4.2 TLBMissQueue 参数化与硬编码清理

`TLBMissQueue` 的 `OUT_free` 统计从固定 4 项改为按 `SIZE` 统一遍历，消除了 `SIZE!=4` 的计数偏差。

```
// src/TLBMissQueue_param.sv:L42-L51
logic[$clog2(SIZE):0] freeCnt;
freeCnt = '0;
for (integer i = 0; i < SIZE; i=i+1)
    if (!queue[i].valid)
        freeCnt = freeCnt + 1'b1;
OUT_free = freeCnt;
if (freeCnt != 0 && OUT_uop.valid)
    OUT_free = OUT_free - 1'b1;
```

与此配套，`Scheduler/StoreQueue/ExternalAXISim/BranchSelector` 中与“4”绑定的路径也转为参数驱动，避免后续扩宽度时再次触发同类问题。

4.4.3 TLB 去重：普通页与 superpage 语义统一

普通页场景下，“vpn 全等 + type 匹配”即可判重；superpage 场景则必须按命中规则使用高位键比较。优化实现将插入判重与命中语义对齐，避免同 superpage 区域重复回填。

```
// src/TLB_fixed_sp_dedup.sv:L99-L121
if (tlb[idx][j].valid &&
    (tlb[idx][j].isSuper == IN_pw.isSuperPage) &&
    (tlb[idx][j].isSuper ?
        (tlb[idx][j].vpn[19-$clog2(LEN):10-$clog2(LEN)] == IN_pw.vpn[19:10]) :
        (tlb[idx][j].vpn == IN_pw.vpn[19:$clog2(LEN)]))
) begin
    already_exists = 1'b1;
end

if (!already_exists)
    tlb[idx][assocIdx].valid <= 1;
```

4.5 外部回归链路与仿真可观测性增强

为了对 SoomRV 的实现开展更充分的测试与功能验证，我们引入了官方 RISC-V 外部测试集 [riscv-tests](#)，用于对 RV32 指令集进行系统性验证。

4.5.1 外部回归入口脚本化

顶层构建系统增加外部测试拉取、构建、smoke、全量回归四类入口，且统一暴露超时与心跳参数。

```
# Makefile:L245-L302
EXTERNAL_TEST_TIMEOUT ?= 180
EXTERNAL_TEST_HEARTBEAT ?= 15

external-tests-run:
    $(PYTHON) scripts/test_suite.py "$(RISCV_TESTS_DIR)/isa" \
        --timeout-sec $(EXTERNAL_TEST_TIMEOUT) \
        --heartbeat-sec $(EXTERNAL_TEST_HEARTBEAT)

external-tests-smoke:
    $(PYTHON) scripts/test_suite.py "$(RISCV_TESTS_DIR)/isa" \
        --categories rv32ui,rv32um,rv32uc --max-tests-per-category 8 \
        --timeout-sec $(EXTERNAL_TEST_TIMEOUT) \
        --heartbeat-sec $(EXTERNAL_TEST_HEARTBEAT)
```

4.5.2 测试退出识别：动态 `.tohost` 地址

外部测试中最隐蔽的问题之一是“程序已写出结束信号，但仿真器未识别地址”。当前实现在 ELF 加载阶段捕获 `.tohost` 段地址，并在测试模式下优先按该动态地址识别结束条件；其中 `COSIM=0` 由 `Top_tb` 侧检测，`COSIM=1` 仍沿用 `simif` 判定路径。

```
// sim/Top_tb.cpp:L759-L766
if (section.name == ".tohost")
    simif.riscvTestTohostAddr = section.addr;
RunChecked(objcopyTool + " -I elf32-little -j " + QuoteShellArg(section.name) + " ...",
    "extract ELF section");
```

```
// sim/Top_tb.cpp:L932-L973
if (args.testMode && wrap->top->clk == 1) {
    auto stUOp = GET(ST_UOp, core->__PVT__SQB_uop.data());
    for (auto tohostAddr : tohostAddrs) {
        if (ExtractStoreWord(stUOp, tohostAddr, data))
            riscvTestReturn = data;
        if (ExtractStoreWord(stUOp, tohostAddr + 4, data) && static_cast<int32_t>(data) ==
0)
            Exit(0);
    }
}

// sim/Simif.cpp:L228-L241 (COSIM 路径)
if (riscvTestTohostAddr != 0) {
    if (phy == riscvTestTohostAddr)
        riscvTestReturn = data;
    else if (phy == (riscvTestTohostAddr + 4) && (int)data == 0)
        return 1;
}
```

这让结束协议从“硬编码地址猜测”升级为“动态地址 + 双路径一致判定”，在 `COSIM=0/1` 下都能稳定识别外部测试退出。这让结束协议从“硬编码地址猜测”变成“跟随 ELF 实际布局”，显著降低了误判风险。

4.5.3 回归脚本鲁棒性

测试执行脚本加入了超时、心跳、已知边界项管理和工具 fallback，解决了“长测无反馈”“失败不可分类”“环境差异导致假失败”的常见问题。

```
# scripts/test_suite.py:L38-L46,L105-L128
UNSUPPORTED_TESTS = {
    "rv32mi-p-pmpaddr": "PMP not implemented/enabled in current SoomRV configuration",
    "rv32mi-p-instret_overflow": "known minstret/cosim semantic mismatch",
}
# --timeout-sec / --heartbeat-sec / --skip-unsupported / --objcopy-fallback
```

4.6 COSIM 语义一致性修复

COSIM 失配处理中，策略是“优先做语义对齐，再评估接口层兼容”，而不是先用仿真侧覆盖去掩盖差异。已落实的关键点包括：

1. `CSR_TINFO` 作为只读零值 CSR 接入，避免读触发非法指令路径。
2. Spike ISA 启用 `zfinx`，与 RTL 浮点能力声明一致。
3. `mstatus.FS/SD` 在 `Zfinx` 路径固定为 0，避免状态位语义分叉。

```
// src/CSR.sv:L585-L596
CSR_tselect,
CSR_tdata1,
CSR_tdata2,
CSR_tdata3,
CSR_tinfo,
CSR_mcontext: rdata = 0;

// sim/Simif.cpp:L92
isa_parser = std::make_unique<isa_parser_t>(
    "rv32imac_zicshr_zfinx_zba_zbb_zbs_zicbom_zifencei_zcb_zihpm_zicntr", "MSU");
```



```
// src/CSR.sv:L926-L929
// Zfinx has FCSR but no FP register file state.
mstatus.fs_ <= 0;
mstatus.sd <= 0;
```

`instret_overflow` 仍归类为计数口径边界项：它反映的是 RTL/对拍标注/Spike 的时点一致性问题，而非普通执行语义错误。

4.7 `--perf` 可观测性与口径固化

性能对比可靠性的前提是计数口径可追溯。当前 `--perf` 机制明确为“按 `minstret` 周期触发 + 差分窗口统计 + 结束时收尾输出”。

```
// sim/Top_tb.cpp:L892-L935
const uint64_t perfInterval = 1024 * 1024 * 8;
uint64_t nextMinstretPerf = wrap->csr->minstret + perfInterval;
...
if (wrap->csr->minstret >= nextMinstretPerf) {
    if (args.logPerformance)
        LogPerf(core);
    nextMinstretPerf = wrap->csr->minstret + perfInterval;
}

// sim/Top_tb.cpp:L783-L795
for (size_t i = 0; i < counters.size(); i++)
    current[i] = counters[i] - lastCounters[i];

double ipc = (double)current[1] / current[0];
double mpki = (double)current[4] / (current[1] / 1000.0);
double bmrte = ((double)current[3] / current[2]) * 100.0;
```

4.8 HardFloat 外部接入工程化

HardFloat 链路从“固定仓库路径依赖”改为“目录参数化 + 自动发现 + 缺失即失败”，从而提升构建可移植性。

```
# Makefile:L8-L14,L123
HARDFLOAT_DIR ?= hardfloat
HARDFLOAT_SRC := $(wildcard $(HARDFLOAT_DIR)/*.v)
ifeq ($(strip $(HARDFLOAT_SRC)),)
$(error No HardFloat Verilog sources found under '$(HARDFLOAT_DIR)')
endif
VERILATOR_CFG = ... -I$(HARDFLOAT_DIR)
```

该改动不改变浮点运算语义，但显著降低了“本地目录结构偶然正确”带来的不确定性。

4.9 Linux 构建与启动链路收敛

Linux 启动问题最终收敛为“配置生效链路 + 产物完整性 + 运行里程碑”三段式验证，而不是只看是否 panic。重点包括：

1. Buildroot 迁移后核对最终 `.config` 与内核 `.config`，而非只看源配置文本。
2. 明确检查 `Image/fw_jump/rootfs.cpio*` 产物是否齐全。
3. 启动日志用固定关键字判定（如 `Run /init as init process`）。

在 Buildroot 2026.02 场景下，针对自定义 OpenSBI Git 源的 hash 阻塞也做了配置侧收敛，使构建链路可持续复现。

4.10 优化与改进的价值

从结果上看，本轮不是“堆功能”，而是把系统演进过程稳定在可验证轨道上：

1. 关键并发路径的隐式行为被协议化。
2. 可复现故障都有对应的最小测试入口。
3. 构建、回归、对拍、启动验证形成闭环。

这为后续性能优化提供了更稳的保障：先保证“解释得清楚”，再追求“跑得更快”。

5 测试与验证

测试体系围绕“可重复、可比较、可解释”三条主线构建。核心问题不是“有没有跑过测试”，而是“结论是否建立在同口径证据上”。

5.1 分层测试策略

我们在原有的测试基础上进行了一些改进，当前验证流程分为五层：

1. 微测试层：验证单个修复点是否命中故障机制。
2. 构建层：验证 `orig/opt` 双实现是否都可稳定编译。
3. 系统层：验证 Linux 与裸机样本上的指标趋势与功能稳定性。
4. 外部回归层：验证通用 ISA 测试集下的通过率与边界项行为。
5. COSIM 层：验证与 Spike 的语义一致性与剩余分歧边界。

这个顺序的意义在于先消除局部不确定性，再做系统结论，避免把多因素耦合噪声误判成优化收益或回退。

5.2 `--perf` 指标定义与口径说明

5.2.1 触发与窗口

`--perf` 由 CLI 参数开启，按 `8 * 1024 * 1024` 指令窗口触发；窗口统计采用 `current = counters - lastCounters` 差分口径，不是全程累计口径。

```
// sim/Top_tb.cpp:L589-L610,L892-L935
case 'p': args.logPerformance = 1; break;
...
if (wrap->csr->minstret >= nextMinstretPerf) {
    if (args.logPerformance)
        LogPerf(core);
}
```

5.2.2 公式与计数器映射

```
// sim/Top_tb.cpp:L787-L795
IPC = instret / cycles
MPKI = mispredicts / (instret / 1000.0)
branch_mispredict_rate = branch_mispredicts / branches * 100
```

对应硬件计数来源：

1. `mcycle`： `cycles`。
2. `minstret`： `instret`。
3. `mhpmcounter[3]`： `branches`。
4. `mhpmcounter[4]`： `branch_mispredicts`。

5. `mhpmcounter[5]`: `mispredicts` 行。

```
// src/CSR.sv:L756-L774
if (!mcountinhibit[2]) minstret <= minstret + ...;
if (!mcountinhibit[3]) mhpmcounter[3] <= mhpmcounter[3] + ...;
if (!mcountinhibit[4] && IN_branchMispr) mhpmcounter[4] <= mhpmcounter[4] + 1;
if (!mcountinhibit[5] && IN_branch.taken) mhpmcounter[5] <= mhpmcounter[5] + 1;
```

需要明确的是: `mispredicts` 这一输出名称是历史遗留命名, 它对应的是 `IN_branch.taken` 事件计数, 不等价于“纯误预测次数”。误判分析应以 `branch mispredicts / branches` 为主。

5.3 模块级定向验证结果

改动主题	测试入口	结果
BT 更新仲裁	make -C test_programs/dev run-bt-arb	RESULT_BT_ARB=PASS
BHT 写后读前递	make -C test_programs/dev compare-bht-fwd	orig FAIL_STALE / bht_fwd PASS
Store 发射节拍	make -C test_programs/dev compare-sqb-issue	orig FAIL_GAP / issue_opt PASS
PageWalker 请求仲裁	make -C test_programs/dev run-pw-arb	RESULT_PAGEWALK_REQ_ARB=PASS
TLBMissQueue 参数化	make -C test_programs/dev compare-tmq-param	orig size8 FAIL / tmq_param size8 PASS
TLB 普通页去重	make -C test_programs/dev compare	orig DUP=1 / fixed DUP=0
TLB superpage 去重	make -C test_programs/dev compare-super	fixed SUPER_DUP=1 / fixed_sp_dedup SUPER_DUP=0
HardFloat 接入smoke	make -C test_programs/dev run-hardfloat	接口链路可独立运行

这组结果覆盖了本轮改造的主要行为修复点, 并且每项均有可重复命令与明确判定语句。

5.4 构建级 A/B 验证约束

在多实现并存架构下, 构建级验证必须串行执行 `clean -> build`, 否则会因共享 `obj_dir` 产生中间文件竞争, 导致与设计改动无关的随机构建失败。

推荐固定流程:

- 1. `make clean`
- 2. `make soomrv <impl=orig ...>`
- 3. `make clean`
- 4. `make soomrv <impl=opt ...>`

该流程的目标是把“构建系统噪声”从评估样本中剥离, 确保 A/B 结果可解释。

5.5 引入外部测试集回归：失败分类与闭环结果

5.5.1 失败分类

基线回归暴露了三类问题：

1. `ld_st` 退出识别缺陷（非功能执行错误）。
2. `instret_overflow` 计数语义分歧（`ERROR 6`）。
3. `pmpaddr` 能力边界（PMP 未实现/未启用）。

这三类问题分别对应协议、口径、能力三个层面，处理策略也必须分层，而不能统一按“功能回归”处理。

5.5.2 修复与脚本策略

`ld_st` 通过动态 `.tohost` 识别闭环；脚本侧通过 `--skip-unsupported` 管理已知边界项，并保留 `--no-skip-unsupported` 强制执行入口。

```
# scripts/test_suite.py:L267-L275,L329-L334
if args.skip_unsupported:
    if t.name in UNSUPPORTED_TESTS:
        skipped_unsupported.append(t)
...
print("skipped unsupported tests:")
```

5.5.3 回归结果

本轮外部回归结果来自统一日志 `docs/logs/external-test-final.log`，执行顺序为“smoke 预检 -> 全量回归”。

1. smoke 预检：`all selected tests passed (18 total)`。
2. 全量回归：`all selected tests passed (198 total)`。
3. 已知边界项：`rv32mi-p-instret_overflow`、`rv32mi-p-pmpaddr` 被显式 `skip`，并附带原因。

全量 `198` 项的类别分布如下（均通过）：

类别	用例数
rv32mi	14
rv32si	6
rv32ui	82
rv32um	16
rv32uc	2
rv32ua	20
rv32uzba	6
rv32uzbb	36
rv32uzbs	16

从日志可见，单测执行时长主要集中在 `0.4s~0.8s` 区间；结合 `--timeout-sec 180` 与 `--heartbeat-sec 15` 的执行参数，当前回归链路已经满足三项工程要求：

1. 结果可复核：通过数、跳过项、类别覆盖都可从单一日志直接重建。
2. 过程可观测：长测期间有心跳，异常可定位到具体类别与用例。
3. 边界可解释：已知不支持项不会污染通过率，同时仍保留强制执行入口用于后续收敛。

5.6 COSIM 一致性验证状态

已闭环项:

- 1. CSR_TINFO 读语义与 Spike 对齐。
- 2. Zfinx ISA 能力声明与 FS/SD 状态位语义对齐。
- 3. 相关 smoke 样本恢复通过。

未闭环项:

- 1. instret_overflow 仍需对齐 RTL 更新时点、对拍标注时点与 Spike 读点。

当前管理策略是默认隔离该边界项，避免污染功能回归信号，同时保留强制模式持续观测。

5.7 Linux 启动与构建链路验证

Linux 验证采用三段式检查:

- 1. 配置生效检查: 最终 .config 是否保留 initramfs 关键项。
- 2. 产物检查: Image/fw_jump/rootfs.cpio* 是否齐全。
- 3. 启动日志检查: 是否出现 Run /init as init process 等关键里程碑。

该流程比“只看是否 panic”更稳定，能够把构建配置偏差与运行时功能问题区分开。

5.8 性能样本解读

本节将性能结论收敛到可复核的数字口径，避免“只看趋势描述”的主观性。所有 Linux 样本来自 docs/logs 下四份 perf 日志，裸机样本来自 baremetal_perf_smoke.log。

5.8.1 Linux --perf 样本规模与全窗口统计

首先给出每份日志的窗口数与全窗口均值/范围:

日志	窗口数	IPC 均值 (min~max)	MPKI 均值 (min~max)	branch mispredict rate 均值 (min~max)
linux_perf_current.log	46	1.159559 (0.802979~2.111755)	15.609377 (2.621054~21.926403)	7.660898% (0.147899%~10.644359%)
linux_perf_current_bp_opt_1.log	12	1.098689 (0.920512~2.111647)	14.529316 (2.619266~18.472791)	6.593743% (0.146273%~9.548904%)
linux_perf_current_final.log	12	1.097209 (0.919038~2.108177)	14.413941 (2.622008~18.367410)	6.544629% (0.147971%~9.499507%)
linux_perf_current_pw_opt.log	12	1.098185 (0.914436~2.108135)	14.481026 (2.622723~18.106580)	6.577399% (0.147334%~9.427187%)

这里必须强调一个解释边界: current 有 46 窗口，而其他日志只有 12 窗口。直接比较“全窗口均值”会混入不同启动阶段比例，结论会偏离同口径对照。因此后续主结论采用“前 12 窗口对齐”。

5.8.2 对齐口径（前 12 窗口）对比

以 current 前 12 窗口为基线，得到三组对照:

对比组	IPC	MPKI	branch mispredict rate
current(前12) -> bp_opt_1	1.095505 -> 1.098689 (+0.2906%)	14.508991 -> 14.529316 (+0.1401%)	6.600536% -> 6.593743% (-0.1029%)
current(前12) -> current_final	1.095505 -> 1.097209 (+0.1555%)	14.508991 -> 14.413941 (-0.6551%)	6.600536% -> 6.544629% (-0.8470%)
bp_opt_1 -> pw_opt	1.098689 -> 1.098185 (-0.0459%)	14.529316 -> 14.481026 (-0.3324%)	6.593743% -> 6.577399% (-0.2479%)

逐窗口方向统计（按对齐窗口逐项比较）：

1. `current(前12)` vs `bp_opt_1`：IPC 8 升 4 降；MPKI 3 升 9 降；误判率 3 升 9 降。
2. `current(前12)` vs `current_final`：IPC 9 升 3 降；MPKI 2 升 10 降；误判率 2 升 10 降。
3. `bp_opt_1` vs `pw_opt`：IPC 5 升 7 降；MPKI 4 升 8 降；误判率 5 升 7 降。

从这三组结果可以得到更稳健的判断：前端相关优化总体未引入明显 IPC 回退，且在误判相关指标上持续给出小幅改善；其中 BHT 写后读前递后的改善方向最一致。

5.8.3 启动阶段扰动下的稳态观察（窗口 3~12）

为了降低最早两个窗口的启动扰动，再看窗口 `3~12` 的均值：

日志	IPC	MPKI	branch mispredict rate
current (3~12)	1.004482	15.706359	7.833367%
bp_opt_1 (3~12)	1.008305	15.732871	7.826360%
current_final (3~12)	1.007344	15.594182	7.767221%
pw_opt (3~12)	1.008520	15.674792	7.806714%

该视角下，四条链路在 IPC 上基本同量级，`current_final` 在 MPKI 与误判率上仍保持更优，说明“方向改善”不是只由启动瞬态造成。

5.8.4 裸机样本：不同负载形态下的吞吐差异

`baremetal_perfc_smoke.log` 中代表性样本如下：

程序	cycles	IPC	MPKI	branch mispredict rate
memcpy.s	8349	3.438496	0.139334	0.097537%
dhry_1.s	294438	2.121554	0.156884	0.085696%
atomic2.s	7862	1.308446	0.874891	0.584226%
atomic.s	7250	0.219448	279.698303	6.015038%
bf.s	7527	1.204464	25.369512	8.702349%

这些数据揭示了两点：

1. 在数据路径友好场景（`memcpy.s`、`dhry_1.s`）下，4 宽 OoO 能稳定给出较高吞吐。
2. 在强保序/高冲突场景（`atomic.s`）下，吞吐显著下降，符合当前实现“优先保证顺序语义与一致性”的设计取向。

此外，短程序（如 `add.s`、`bitmanip.s`）会出现极端 stall 百分比，这与 5.2 节的分母口径一致，不能直接作为系统级性能结论。

6 总结与展望

6.1 已完成的阶段性目标

1. 前端预测路径完成了 BT 更新仲裁协议化与 BHT RAW 前递修复，收敛行为更稳定。
2. LSU 与 MMU 慢路径完成了发射节拍、请求仲裁、参数化与去重一致性修复。
3. 外部回归、COSIM、Linux 启动验证形成可执行闭环，测试结果具备可解释性。
4. 构建系统支持多实现开关化 A/B 验证，回滚与定位成本显著下降。

6.2 仍需持续推进的问题

1. `instret_overflow` 仍缺少统一的时点语义模型。
2. BTB 直接映射在热点别名场景下仍可能造成冲突压力。
3. 高保序路径（原子、CSR/SC）在极端负载下的吞吐仍偏保守。
4. Linux 性能统计仍需更长窗口和更多阶段样本支撑。

6.3 后续工作建议

1. 建立 `instret_overflow` 专项对拍模型，统一 RTL/Spike/testbench 的计数时点语义。
2. 为 BTB/TAGE、PageWalker/TLB 增加冲突与回放统计，形成“结构瓶颈 -> 指标变化”的因果链。
3. 把 Linux 启动验证扩展为自动化流水线步骤，覆盖构建、产物、启动里程碑和 `perf` 摘要。
4. 在保证协议语义不回退的前提下，逐步评估保序路径的局部放宽空间。

7 成员分工与贡献

卫佳乐

1. 负责范围：前端取指、预测恢复、BTB/TAGE/ReturnStack。
2. 贡献点：统一前端改向语义，补齐 `fetchLimit` 风险分析，梳理 `predIllegal` 修复路径。

刘元昊

1. 负责范围：Decode/Rename/Issue/Load 读操作数链。
2. 贡献点：明确 `tag/sqN` 契约、IssueQueue 发射条件与写回冲突处理。

何国真

1. 负责范围：执行单元、ROB 提交、CSR/Trap 控制。
2. 贡献点：明确提交门控与提交侧预测更新策略，完成 CSR 兼容性收敛。

陈冠宇

1. 负责范围：AGU/LSU/SQ/SQB/TLB/PageWalker。
2. 贡献点：梳理访存快慢路径、TLB miss 协同、内存序回放机制。

刘卓敏

1. 负责范围：Top/SoC/Core/memc 等各个模块的串联，工具链更新与兼容性修复等。
2. 贡献点：统一接口口径，对新版本 Linux 构建测试，对项目改进与优化，完善文档等。